

LINMA1170 – Devoir 1 : Factorisation LU

V. Degrooff, C. Heneffe, N. Tihon, J.-F. Remacle

2025–2026

Contexte

Dans ce devoir, vous allez implémenter deux décompositions fondamentales de l'algèbre linéaire. La factorisation LU pour des matrices carrées réelles et la factorisation de Cholesky pour des matrices réelles symétriques.

Implémentations

On vous demande d'écrire les deux fonctions suivantes dans un fichier `devoir_1.c` :

1. Une fonction qui calcule la factorisation LU d'une matrice A avec un pivotage complet. Contrairement au cas du pivotage partiel, le pivot à l'étape k est choisi comme étant la plus grande entrée de la sous-matrice $A_{k:n,k:n}$. Cette stratégie est plus coûteuse, mais améliore la stabilité de l'algorithme. En forme matricielle, cela revient donc à appliquer des matrices de permutation P_k à gauche, mais aussi des permutations Q_k à droite. On a alors

$$PAQ = LU.$$

```
int lu_factorisation(double *A, int *P, int *Q, int n);
```

qui prend comme arguments:

- le tableau A qui contient la matrice A en mémoire. Notez que cette matrice est stockée en *row-major* dans un vecteur unidimensionnel. En sortie de fonction, le tableau A a été remplacé par sa factorisation LU.
 - les tableaux P et Q , de taille n , non initialisés, qui vous seront utiles pour effectuer vos permutations sur les lignes/colonnes de A .
 - l'entier n qui représente le nombre d'inconnues.
2. Une fonction qui calcule la *fast Cholesky factorisation* d'une matrice SDP, en utilisant l'algorithme de Strassen pour effectuer les produits matriciels.

```
int fast_Cholesky(double *A, int n);
```

- le tableau A contient la matrice A en mémoire. Notez que cette matrice est stockée en *row-major* dans un vecteur unidimensionnel de taille n^2 . En sortie de fonction, le tableau A a été remplacé par sa factorisation de Cholesky. Étant donné la symétrie de la matrice, vous avez la possibilité d'accélérer votre algorithme en ne travaillant qu'avec la partie triangulaire inférieure/supérieure de A .
- l'entier n qui représente le nombre d'inconnues.

Analyses numériques

- Analysez la complexité temporelle de vos implémentations LU et Cholesky. Présentez vos résultats temps/taille sur un plot **log-log**. Comparez ces résultats à ceux de la librairie **SciPy** ou **Lapack**.
- Analysez le *growth factor* $\rho = \frac{\max_{i,j} |U_{ij}|}{\max_{i,j} |A_{ij}|}$ sur des matrices aléatoires de tailles variées, et ce pour différentes stratégies : *no pivoting*, *partial pivoting* and *total pivoting*.
- (Bonus) Reprenez la matrice de l'opérateur Laplacien 2D discrétisé par différences finies sur un carré, avec des conditions frontières de Dirchlet (cf. APE S2-S3). Mettez en évidence le phénomène de *fill-in*, c'est-à-dire l'augmentation du nombre d'entrées non-nulles dans le tableau **A**. Montrez l'impact de la numérotation des noeuds sur le *fill-in*. Quel serait le pire cas possible ? Pour cette question, n'effectuez aucun *pivoting* durant la factorisation.

En bref

On vous demande de soumettre **par groupe de deux** sur Gradescope :

- pour le **16 mars 2026 à 14h**.
 - votre fichier de code **devoir_1.c** qui contient au moins les fonction demandées
 - * La seule librairie externe admise, mais non obligatoire, est la librairie **BLAS**
 - le script (.py probablement) qui reproduit les résultats/figures de votre présentation
- pour le **23 mars 2026 à 12h**
 - les slides de votre présentation, en format **.pdf**.

Toutes les implémentations seront soumises à un logiciel anti-plagiat.